Lab: Confluent Kafka in KRaft Mode on a Local VM

Lab Objectives

By the end of this lab you will:

- Install Confluent Platform on a local Linux VM.
- Configure and run a **single-broker KRaft cluster** (combined controller + broker).
- Configure and run a **multi-broker KRaft cluster** (1 controller + 3 brokers) on the same VM.
- Create topics, produce, and consume messages against both setups.

---

0. Prerequisites and Setup

1. **VM & OS**
    - 1 Linux VM (4 GB RAM+ recommended).
    - Java 11 or 17 installed and on the PATH (java -version).
2. **Download & Install Confluent Platform**
    - Download the latest Confluent Platform tar/zip from Confluent's on-prem installation page (community or enterprise).
    - On the VM:

```
# Example install location
cd /opt
tar -xzf confluent-<version>.tar.gz
ln -s confluent-<version> confluent
export CONFLUENT_HOME=/opt/confluent
export PATH=$CONFLUENT_HOME/bin:$PATH
```

    - All following commands assume you run them with CONFLUENT_HOME set.

3. **Create a Working Directory**

```
cd $CONFLUENT_HOME
mkdir -p my-kraft-labs
```

Part A: Single-Broker Cluster in KRaft (Combined Mode)

In this part you run **one process** that acts as both controller and broker (combined mode) for local development.

Step A1: Copy the Base KRaft Configuration

Recent Confluent Platform distributions ship a KRaft-ready server config under etc/kafka/kraft/server.properties.

```
cd $CONFLUENT_HOME

cp etc/kafka/kraft/server.properties my-kraft-labs/kraft-single.properties
```

Step A2: Configure Combined Controller + Broker

Edit my-kraft-labs/kraft-single.properties and ensure the following key settings (leave other defaults as-is):

```
############################## KRaft Basics ##############################

# Run as both controller and broker (combined mode – dev only)
process.roles=broker,controller

# Unique ID for this node
node.id=1

# Static controller quorum, single controller on this same node
controller.quorum.voters=1@localhost:9093

############################## Listeners ##############################

# Client (broker) listener and controller listener
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
```

```
# Names used for controller and broker listeners
controller.listener.names=CONTROLLER
inter.broker.listener.name=PLAINTEXT

# Security protocol mapping
listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

############################# Logs #############################

log.dirs=/tmp/kraft-single-logs
```

Notes:

- process.roles activates KRaft mode and defines which roles this JVM plays.
- node.id must be unique in a cluster; here there's only one node.
- controller.quorum.voters lists the controllers in the Raft quorum; for a single-node dev cluster it only contains this node's id and controller listener.

Step A3: Generate a Cluster ID

You must generate a single **cluster ID** and reuse it for all nodes in a cluster.

```
cd $CONFLUENT_HOME
export KAFKA_CLUSTER_ID="$(kafka-storage random-uuid)"
echo "Cluster ID: $KAFKA_CLUSTER_ID"
```

Step A4: Format the KRaft Storage Directory

Format KRaft metadata and log directories **once** before starting the node.

```
kafka-storage format \
  --cluster-id $KAFKA_CLUSTER_ID \
  --config my-kraft-labs/kraft-single.properties
```

If you change KRaft-critical settings or log directories later, you must delete the log directory and re-run kafka-storage format.

Step A5: Start the Single-Broker KRaft Node

Open **Terminal 1**:

```
cd $CONFLUENT_HOME

kafka-server-start my-kraft-labs/kraft-single.properties
```

Watch the logs:

- The node should start in KRaft mode as both broker and controller.
- It should open client listener on :9092 and controller listener on :9093.

Leave this terminal running.

Step A6: Create a Test Topic

Open **Terminal 2**:

```
cd $CONFLUENT_HOME


kafka-topics \
  --create \
  --topic cp-kraft-single-test \
  --bootstrap-server localhost:9092 \
  --partitions 1 \
  --replication-factor 1
```

Verify:

```
kafka-topics --list --bootstrap-server localhost:9092

kafka-topics --describe --topic cp-kraft-single-test --bootstrap-server localhost:9092
```

You should see one partition and replication factor 1.

Step A7: Produce and Consume Messages

1. **Consumer (Terminal 3)**

```
cd $CONFLUENT_HOME
kafka-console-consumer \
  --topic cp-kraft-single-test \
  --from-beginning \
  --bootstrap-server localhost:9092
```

2. **Producer (Terminal 4)**

```
cd $CONFLUENT_HOME
kafka-console-producer \
  --topic cp-kraft-single-test \
  --bootstrap-server localhost:9092
```

Type a few lines:

First message (KRaft single node)

Second message

They should appear in the consumer terminal almost immediately.

Step A8: Stop and Reset (Optional)

- Stop producer and consumer with Ctrl+C.
- Stop the KRaft node in Terminal 1 with Ctrl+C.
- To fully reset this single-broker lab:

```
rm -rf /tmp/kraft-single-logs
```

You can now move on to the multi-broker lab.

Part B: Multi-Broker KRaft Cluster on a Single Machine

In this part, you create a **small KRaft cluster** on one VM with:

- 1 dedicated **controller** node (process.roles=controller)
- 3 dedicated **broker** nodes (process.roles=broker)

- All processes running as separate JVMs but sharing one cluster ID.

Step B1: Prepare Config Files

Create a subdirectory and copy the base KRaft config:

```
cd $CONFLUENT_HOME

mkdir -p my-kraft-labs/multi

cd my-kraft-labs/multi


cp $CONFLUENT_HOME/etc/kafka/kraft/server.properties controller.properties

cp $CONFLUENT_HOME/etc/kafka/kraft/server.properties broker-1.properties

cp $CONFLUENT_HOME/etc/kafka/kraft/server.properties broker-2.properties

cp $CONFLUENT_HOME/etc/kafka/kraft/server.properties broker-3.properties
```

Confluent's multi-broker tutorial takes a similar approach by preparing one controller config and multiple broker configs on the same host.

Step B2: Configure the Controller Node

Edit controller.properties:

```
############################## KRaft Basics ##############################


process.roles=controller

node.id=10


# Static quorum with a single controller (for dev)

controller.quorum.voters=10@localhost:9097


############################## Listeners ##############################


listeners=CONTROLLER://:9097

controller.listener.names=CONTROLLER
```

listener.security.protocol.map=CONTROLLER:PLAINTEXT

inter.broker.listener.name=PLAINTEXT

############################## Logs ##############################

log.dirs=/tmp/kraft-controller-logs

Notes:

- node.id=10 is unique and appears in controller.quorum.voters because this node is a controller.
- Only controller listeners are configured here; this node will not accept client traffic.

Step B3: Configure Three Broker Nodes

broker-1.properties

process.roles=broker

node.id=1

Client listener for this broker

listeners=PLAINTEXT://:9092

advertised.listeners=PLAINTEXT://localhost:9092

Same controller quorum as controller.properties

controller.quorum.voters=10@localhost:9097

controller.listener.names=CONTROLLER

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

inter.broker.listener.name=PLAINTEXT

log.dirs=/tmp/kraft-broker-logs-1

broker-2.properties

process.roles=broker

node.id=2

listeners=PLAINTEXT://:9093

advertised.listeners=PLAINTEXT://localhost:9093

controller.quorum.voters=10@localhost:9097

controller.listener.names=CONTROLLER

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

inter.broker.listener.name=PLAINTEXT

log.dirs=/tmp/kraft-broker-logs-2

broker-3.properties

process.roles=broker

node.id=3

listeners=PLAINTEXT://:9094

advertised.listeners=PLAINTEXT://localhost:9094

controller.quorum.voters=10@localhost:9097

controller.listener.names=CONTROLLER

listener.security.protocol.map=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

inter.broker.listener.name=PLAINTEXT

log.dirs=/tmp/kraft-broker-logs-3

Important details:

- All brokers reference the same controller quorum (10@localhost:9097) but their own node.id does **not** appear in controller.quorum.voters, because they are broker-only nodes.
- Each broker has a unique listener port and its own log.dirs, so they can all run on the same machine.

Step B4: Generate a Cluster ID (Multi-Broker)

From $CONFLUENT_HOME:

```
cd $CONFLUENT_HOME

export MULTI_CLUSTER_ID="$(kafka-storage random-uuid)"

echo "Multi-broker Cluster ID: $MULTI_CLUSTER_ID"
```

You must use the **same** cluster ID to format the controller and all brokers.

Step B5: Format Storage for Controller and Brokers

From $CONFLUENT_HOME:

```
# Controller

kafka-storage format \

  --cluster-id $MULTI_CLUSTER_ID \

  --config my-kraft-labs/multi/controller.properties


# Brokers

kafka-storage format \

  --cluster-id $MULTI_CLUSTER_ID \

  --config my-kraft-labs/multi/broker-1.properties
```

```
kafka-storage format \
  --cluster-id $MULTI_CLUSTER_ID \
  --config my-kraft-labs/multi/broker-2.properties

kafka-storage format \
  --cluster-id $MULTI_CLUSTER_ID \
  --config my-kraft-labs/multi/broker-3.properties
```

The Confluent docs stress that kafka-storage is run **once per node**; if you misconfigure something, you must delete the log directories and re-run format.

Step B6: Start the Controller, Then the Brokers

1. **Controller (Terminal 1)**

```
cd $CONFLUENT_HOME
kafka-server-start my-kraft-labs/multi/controller.properties
```

Wait until logs show the controller quorum is ready (single-node quorum).

2. **Brokers (Terminals 2–4)**

```
# Terminal 2
cd $CONFLUENT_HOME
kafka-server-start my-kraft-labs/multi/broker-1.properties

# Terminal 3
cd $CONFLUENT_HOME
kafka-server-start my-kraft-labs/multi/broker-2.properties

# Terminal 4
cd $CONFLUENT_HOME
kafka-server-start my-kraft-labs/multi/broker-3.properties
```

Each broker should connect to the controller at localhost:9097, register itself, and log that it joined the cluster.

(Optional) Inspect metadata quorum:

```
cd $CONFLUENT_HOME
kafka-metadata-quorum \
  --bootstrap-controller localhost:9097 \
  describe
```

This shows the KRaft metadata partition, leader, and voter information.

Step B7: Create a Replicated Topic

Use **any broker** as the bootstrap server (we'll use localhost:9092):

```
cd $CONFLUENT_HOME

kafka-topics \
  --create \
  --topic cp-kraft-multi-test \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 3
```

Verify:

```
kafka-topics \
  --describe \
  --topic cp-kraft-multi-test \
  --bootstrap-server localhost:9092
```

You should see:

- PartitionCount: 3
- ReplicationFactor: 3

- Each partition's Replicas: and Isr: lists containing broker ids 1,2,3 (order may vary).

Step B8: Produce and Consume in the Multi-Broker Cluster

1. **Consumer (Terminal 5)**

```
cd $CONFLUENT_HOME
kafka-console-consumer \
  --topic cp-kraft-multi-test \
  --from-beginning \
  --bootstrap-server localhost:9092
```

2. **Producer (Terminal 6)**

```
cd $CONFLUENT_HOME
kafka-console-producer \
  --topic cp-kraft-multi-test \
  --bootstrap-server localhost:9093
```

Type:

Message from broker on 9093

Another message in multi-broker KRaft

These should appear exactly once in the consumer terminal, regardless of which broker you connect to, because metadata and replication are coordinated through the KRaft controller.

You can repeat the test using --bootstrap-server localhost:9094 to show that any broker in the cluster can serve as entry point.

Step B9: Stop and Clean Up the Multi-Broker Cluster

1. Stop producer and consumer (Ctrl+C).
2. Stop all three brokers (Ctrl+C in each broker terminal).
3. Stop the controller (Ctrl+C in controller terminal).
4. Delete logs to fully reset:

```
rm -rf /tmp/kraft-controller-logs \
    /tmp/kraft-broker-logs-1 \
    /tmp/kraft-broker-logs-2 \
    /tmp/kraft-broker-logs-3
```

If you want to recreate the cluster, repeat Steps B4–B8 with a new or reused MULTI_CLUSTER_ID.